Backups — Cloudflare R2

Everything the desk records is copied off the box every weekday night to a Cloudflare R2 bucket: as a **backup** (to restore a lost or damaged database) and as **analysis files** (register CSVs that DuckDB, pandas or Excel read directly). The GitHub backup that came before it is gone — it pushed API keys in plain text (S-2).

Code	`scripts/backup-r2.js` (backup), `scripts/restore-r2.js` (restore), `src/backup/r2.js` (the R2 client), `scripts/set-r2-keys.sh` (credentials)
Runs	under pm2 as `r2-backup`, **weekdays 23:20 UTC**, and once at every deploy
Credentials	`data/r2.json` on the box, mode 0600, never committed, never backed up
Bucket	`trade-desk-backup` (set in `data/r2.json`)
Size	about 17 MB compressed per night (measured 2026-10-02)
Tests	`tests/backup.r2.test.js` (against a fake S3 server)

What is in the bucket

```
trade-desk-backup/
├── daily/
│   └── 2026-10-02/                      one folder per night, by New York date
│       ├── manifest.json                what was copied, sizes, sha256, failures
│       ├── data/tradedesk.db.gz         T1
│       ├── data/t2.db.gz … data/t11.db.gz   every tool, running or archived
│       ├── data/<shared>.json.gz        industry map, O'Neil files, shortlist, registry…
│       ├── data/journal/journal.db.gz   the journal
│       └── qp/platform.db.gz            qp's strategies and backtests
├── monthly/
│   └── 2026-10/                         the first COMPLETE night of each month
│       ├── manifest.json                written last — the "month is kept" marker
│       └── …same files as a daily folder…
└── analysis/
    └── T2/R1/2026-10-02.csv.gz          one CSV per tool × register × day
        T2/R4A/2026-10-02.csv.gz
        …                                registers: R1 R2 R3A R3B R4A R4B Shortlist
```

daily/

Every SQLite database on the box, each gzipped, plus the shared `data/*.json` files:

Source on the box	Name in the bucket
`data/tradedesk.db` (T1), `data/t2.db` … `data/t11.db`	`data/<same>.db.gz`
`data/*.json` except the secret files below	`data/<same>.json.gz`
`data/journal/journal.db`	`data/journal/journal.db.gz`
`~/.qp/platform.db` (`QP_DB` moves it)	`qp/platform.db.gz`

Stopped and archived tools are included: their databases are the only copy of what they collected.

Consistent copies. Databases are copied with SQLite's online backup API, so a copy taken while a tool is writing is still a whole database, never half a write.

manifest.json

One per folder:

```
{
  "date": "2026-10-02",
  "startedAt": "2026-10-02T23:20:01.512Z",
  "finishedAt": "2026-10-02T23:21:14.088Z",
  "host": "ip-172-31-19-99",
  "files": [
    { "name": "data/t2.db", "kind": "sqlite", "bytes": 1834221, "sha256": "…" }
  ],
  "analysis": [ { "key": "analysis/T2/R1/2026-10-02.csv.gz", "rows": 38 } ],
  "failures": [],
  "notes": [ "T3 R1: dates unreadable (…)" ]
}
```

`failures` is anything that should have been copied and was not. `notes` are not failures: a tool that is not running, or a register with no rows that day — the database copy already holds everything that tool has.

monthly/

The first night of each month whose **every** file arrived is copied to `monthly/<YYYY-MM>/`, and kept for good.

What if that night fails? The month's own `manifest.json` is written last, and only when the night had zero failures. Each night checks for it: while it is missing, that night's copy is written

to the month again. So a failed or partial night is simply retried the next night, and the next, until one completes — the month is never marked kept with a file missing.

analysis/

Each tool's registers for the day, as CSV, read from each tool's own `/api/warehouse` (the same answers its pages and qp get — live tools and the archive alike). One file per tool × register × day: `analysis/<tool>/<register>/<date>.csv.gz`. Nested values are JSON in the cell. `node scripts/backup-r2.js --all-dates` exports every past day as well (run it once after setting the backup up, or after a tool's history was restored).

Read them in place, for example with DuckDB:

```
SELECT * FROM read_csv_auto('analysis/T2/R4A/*.csv.gz');
```

What is never in the bucket

File	Why
`data/keys.json`	API keys (Alpaca, Polygon, …)
`data/r2.json`	the backup's own credentials
`data/journal/journal-accounts.json`	the journal's Alpaca account keys (only `journal.db` is taken from that folder)
`data/auth.json`	the login's password hash and signing secret

A backup must not be the place a key leaks from. Keep the keys themselves somewhere safe of your own; after a restore, set them again (Settings, the journal's accounts, `set-r2-keys.sh`, `set-password.sh`).

Not backed up, because they rebuild themselves:

What	Where	How it comes back
Bar and data caches	`~/.qp-cache/`	re-fetched on demand and by the nightly qp job
Trained score models	`src/scoring/outputs*`	retrain from the backed-up registers (tool → Analysis → train)
qp operations log	`quant-platform/chart/data/oplog-*.jsonl`	a log, not data — lost history only
The code	the repository	`git clone`

Retention

Set on the bucket, not in code: an R2 **lifecycle rule** on the `daily/` prefix deletes each night's folder after the number of days set there (60 is the recommendation: two months of nightly restore points). `monthly/` and `analysis/` have no rule and are kept. Because a month is only marked kept after a complete night (above), a short daily retention can never cost a month. Cloudflare dashboard → R2 → `trade-desk-backup` → Settings → Object lifecycle rules.

Checking it

- **On the box:** `pm2 logs r2-backup --lines 20 --nostream`. A good night ends: `[r2-backup] 2026-10-02: 19 file(s), 17.0 MB compressed; N register export(s); 0 failure(s)`. pm2 marks the run errored (exit code 1) when anything failed.
- **The deploy** prints `R2 BACKUP — nightly at 23:20 UTC (Mon–Fri), and once now`, or says it is off when `data/r2.json` is missing.
- **In the bucket:** a `daily/<today>/manifest.json` with `"failures": []`.
- **Run it by hand:** `node scripts/backup-r2.js` (or `--dry-run` to build everything and upload nothing).

Restoring

Restore never writes over live data. Files land in `restore/<date>/` (gitignored), unzipped:

```
cd ~/Tade-desk-server
node scripts/restore-r2.js 2026-10-02                # list what that night holds
node scripts/restore-r2.js 2026-10-02 data/t2.db     # download one file
node scripts/restore-r2.js 2026-10-02 all            # download everything
node scripts/restore-r2.js monthly/2026-10 all       # from a monthly copy
```

Putting a file back is a deliberate step of your own:

```
pm2 stop tool-T2                                 # the process that owns the file
cp restore/2026-10-02/data/t2.db data/t2.db
rm -f data/t2.db-wal data/t2.db-shm              # stale journal files of the old copy
pm2 start tool-T2
```

Owners: `tool-T<n>` for a tool's database (T1's is `tradedesk.db`), `archive` for an archived tool's, `journal` for the journal, `qp` for `platform.db`.

A new box from nothing

1. Clone the repository and run `./deploy-tools.sh` (it starts empty).
2. `bash scripts/set-r2-keys.sh`, then `node scripts/restore-r2.js <date> all`.
3. `pm2 stop all`, copy `restore/<date>/data/*` into `data/` and `restore/<date>/qp/platform.db` to `~/.qp/platform.db`, `pm2 start all`.
4. `./scripts/set-password.sh`; put the API keys back in Settings and the journal's accounts.

Credentials

`bash scripts/set-r2-keys.sh` asks for the R2 endpoint, the access key id, the secret (typed hidden) and the bucket, writes `data/r2.json` at mode 0600 and tests them with a read-only request (nothing is written). Use an **Account API token** scoped to this bucket with *Object Read & Write*. To rotate: create a new token in Cloudflare, run the script again, delete the old token.